

Schema-Aware Candidate SQL Selection

项目阶段性汇报

Dongx Xia Student ID: 50062682

2026 年 9 月

摘要

本项目研究 Text-to-SQL 系统中的候选选择问题：在上游模型已经生成若干条 SQL 后，如何从中选出真正符合问题语义的一条。项目最初计划训练新的 Schema Linker，提高表和列的预测准确率；在分析候选池后，研究重点转向了一个更容易独立验证的问题，即保持原 Schema Linker 和候选生成器不变，训练监督式 Schema-aware reranker。当前已经完成 BIRD 数据处理、固定候选池构建、候选执行标注、结构与 schema 特征提取、GPU pairwise MLP、冻结 MiniLM 语义表示、残差融合和置信门控。现有 50 题测试中，自一致性准确率为 62%，候选池上界为 70%；当前模型最高只能追平基线，schema-aware 版本最高为 60%。这个结果没有支持最初的性能假设，但揭示了更具体的问题：有效正负候选对过少，简单 schema 特征在同题候选之间缺乏差异，而且验证与执行标签仍有噪声。下一阶段将优先修复数据和评估协议，再训练真正面向 SQL 语义差异的 reranker。

1 问题从哪里来

Text-to-SQL 系统通常不只生成一条答案。通过不同提示、采样或搜索路径，可以得到多条语法正确且能够执行的 SQL。常见做法是按执行结果聚类，选择最大结果簇中的候选。这种自一致性方法简单而稳定，但它依赖一个并不总成立的假设：多数候选对应的结果更可能正确。如果生成器对问题产生了同一种误解，例如选错相似列、遗漏过滤条件或采用错误连接，多条错误 SQL 也会形成最大的结果簇。

项目开始时，我更关注上游 Schema Linking，计划用神经网络预测问题涉及的表和列。然而，这会同时改变候选的生成分布，很难判断最终增益究竟来自 linker、生成器还是选择器。对固定候选池的检查给出了更直接的切入点：50 个开发集问题上，自一致性只选对 31 题，而候选池中有 35 题至少包含一条正确 SQL。也就是说，现有候选中已经有答案，但选择规则浪费了 4 题、8 个百分点的空间。

因此，项目最终聚焦于一个独立问题：给定问题、数据库 schema 和固定候选集合，训练一个 selector 对候选重新排序。这个定位使候选生成能力和候选选择能力可以分别测量，也避免把项目结果归因于上游搜索预算的变化。

2 文献带来的主要启发

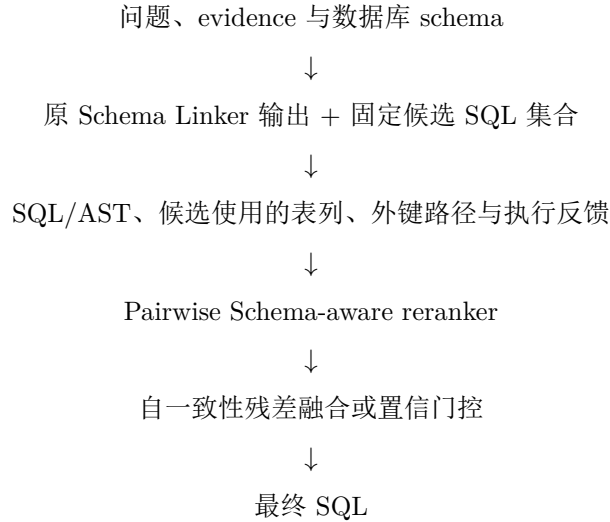
早期工作主要帮助我理解 schema 信息应该如何进入模型。SLSQL 的分析表明，Schema Linking 对跨数据库泛化非常关键；在提供高质量 linking 时，一个相对简单的生成模型也能得到较强结果 [2]。这促使我保留原 linker 输出，但由于监督式 linker 本身也并不完美，它不能成为硬过滤条件。

BRIDGE 将问题、schema 和问题中出现的数据库值放进统一序列，通过 BERT 学习文本与表格结构之间的依赖 [1]。RAT-SQL 则进一步把问题到表列的匹配、表列包含关系以及主外键关系编码为显式关系 [3]。这两项工作说明，schema-aware 不应只等价于字符串重合率；字段语义、关系图和问题上下文需要联合建模。当前实现中的 candidate-used tables、columns、foreign-key connectivity 和 join completeness 都来自这部分启发。

较新的工作使研究重点转向候选选择。MSc-SQL 让 critic 同时观察多条候选及其执行元数据，在固定候选池上将 BIRD Dev 的结果从 self-consistency 的 62.6% 提高到 65.6%，而候选池 oracle 为 71.4% [4]。这说明 selector 本身确实有可优化空间。CHASE-SQL 使用经过训练的 pairwise 比较模型，将所有候选两两比较并累计胜出分数 [5]，直接启发了本项目的 pairwise RankNet 目标。DPC 则从另一个方向指出，模型只看 SQL 文本时容易受到确认偏差和符号推理能力限制；它通过最小区分数据库和 SQL/Python 双范式验证来暴露候选间的逻辑差异 [6]。这一思路尚未实现，但为后续构造更可靠的难例和执行验证提供了方向。

3 最终研究方案

本项目不重新训练 Schema Linker，也不要求候选来自某一种特定搜索算法。上游只负责提供固定候选池，selector 负责回答“哪一条更值得选”。整体流程如下：



候选表示分为四部分。第一部分是 SQL 结构，包括表列数量、谓词、聚合、分组、排序、子查询和解析状态。第二部分是 schema 信息，包括候选实际使用的表列、与 linker 输出的覆盖关系、未链接元素、外键连通性和 join key。第三部分是执行信息，包括执行成功、结果行列数、空值比例和执行结果簇大小。第四部分是可选的生成轨迹信息，例如路径深度和已有一致性分数。

对于同一问题的一条正确候选 s^+ 和错误候选 s^- ，模型学习

$$f(q, s^+) > f(q, s^-),$$

并使用 RankNet 损失

$$\mathcal{L}_{\text{rank}} = -\log \sigma(f(q, s^+) - f(q, s^-)).$$

推理时不必无条件替换自一致性。当前实现可以将神经分数作为 residual 加到结果簇分数上，也可以只在模型相对基线候选具有足够大置信差时改票。这一设计的目的，是在保留基线稳定性的同时尝试恢复少数 oracle-gap 问题。

4 已经完成的实现

项目已经形成从数据到评估的完整离线流程。BIRD train/dev 被安装到本地后，脚本可以按数据库采样问题、生成候选、保存中间结果，并在中断后继续运行。为了尽快获得足够多的训练问题，后期使用直接候选生成绕开了完整 MCTS；这不会改变研究目标，因为所有 selector 都在相同的离线候选池上比较。

目前实现的模型包括 Logistic Regression、小型 GPU MLP 和冻结的 MiniLM 编码器。数值模型的 SQL 加执行版本有 25 个特征，加入 schema 后为 40 个，再加入 MCTS 信息后为 44 个。语义版本将 “question + evidence” 与 “candidate SQL + candidate schema + linked schema” 组成文本对，并将冻结编码器的表示与数值特征拼接。模型训练、checkpoint、逐候选分数和汇总报告均已保存。

表 1: 当前候选池的实际规模。

项目	数量
请求生成的训练问题	300
成功保存候选的问题	286
可执行 gold 并进入训练统计的问题	269
真实训练候选	903
至少包含一条正确候选的问题	213
同时包含正确和错误候选的问题	40
测试问题 / 测试候选	50 / 186

这里最值得注意的是最后第三行。903 条候选看起来并不少，但真正能够形成问题正负排序监督的只有 40 个问题。大量问题的候选要么全部执行正确，要么全部错误，因此不能训练“同一个问题中哪一条更好”。报告中的实际训练 pair 只有约 111–115 个。

5 实验结果

所有方法在同一组 50 题、186 条候选上评估。Self-consistency 选对 31 题，候选池 oracle 为 35 题。换言之，15 题根本没有正确候选，31 题已经被基线选对，真正可能由 selector 恢复的只有 4 题。

目前没有模型超过 62% 的基线。最好的 SQL-only 模型追平基线，最好的 schema-aware 模型为 60%。在一个代表性的 schema-aware 结果中，模型成功修正了 2 题，但把原本正确的 3 题改错，因此净结果从 31/50 下降为 30/50。这个结果说明模型并非完全不会识别错误候选，但它的置信度还不足以支持稳定改票。

表 2: 固定测试候选池上的主要结果。

方法	EX	相对基线
Execution self-consistency	62%	–
Candidate oracle	70%	+8
数值 MLP, SQL + execution	60%	–2
数值 MLP, schema-aware	58–60%	–2 至 –4
冻结 MiniLM, SQL + execution	60–62%	0 至 –2
冻结 MiniLM, schema-aware	58–60%	–2 至 –4
Schema + execution + MCTS	58–60%	–2 至 –4

6 结果如何理解

第一个问题是监督数据的有效规模。Pairwise reranker 需要同一问题中的正负候选，而当前 269 个问题中只有 40 个满足条件。继续增加“全部错误”或“全部正确”的问题，对排序训练帮助很小。下一轮数据构建应该把目标从候选总数改为正负共存问题数。

第二个问题是当前 schema 特征缺乏区分度。在 40 个正负混合训练问题中，正确和错误候选的 table coverage 在 36 题中完全相同，column coverage 在 26 题中相同，foreign-key connectivity 在 39 题中相同。很多错误 SQL 与正确 SQL 使用相同表列，只在过滤、聚合或排序上不同。由此可见，简单的 schema overlap 无法解决候选选择问题。

第三个问题是错误候选也可能非常符合 linker。候选通常由相同问题和相同 linker 输出生成，系统性错误会沿着同一 schema 传播。因此，“与 linker 一致”不能直接解释为“语义正确”。真正有用的 schema-aware 模型需要理解问题短语与候选操作之间的对应关系，例如“最高”是否落实为正确的排序和限制，“平均”作用于哪一列，以及过滤条件是否落在正确实体上。

第四个问题是当前所谓的语义模型仍然较弱。MiniLM 原本针对文本检索训练，而且在本实验中完全冻结。它提供的是通用相关性表示，并没有针对 SQL 运算、聚合、连接和执行状态进行训练。因此，现有结果只能说明冻结的通用语义特征没有帮助，不能据此否定候选级语义建模。

最后，实验协议还有两处需要修正。Gold-positive 消融目前在划分训练和验证集之前注入 gold，导致验证集中也出现人工候选，内部验证分数因此明显虚高。执行标签还会无条件忽略结果行顺序，可能把错误的 ORDER BY 候选标为正确。这两项问题都应在扩大实验前修复。

7 下一阶段计划

下一阶段不会首先扩大模型，而是按以下顺序推进：

1. **修复评估协议。**先按数据库划分，再只向训练部分注入 gold；接入官方 BIRD evaluator，并正确处理 ORDER BY。
2. **扩大有效排序数据。**目标是至少 500 个正负共存问题，并定向构造错误列、错误 join、漏

条件、错误聚合和错误排序等可执行难例。

3. **训练真正的 SQL 语义模型。**输入问题、evidence、候选 AST、候选使用的具体表列描述和外键路径，比较冻结、部分解冻和完整微调。
4. **建模候选之间的差异。**从独立打分发展到 pairwise tournament 或 groupwise critic，显式指出两条 SQL 在谓词、聚合、连接和排序上的差别。
5. **采用保守决策。**在数据库隔离验证集上校准门控阈值，主要报告正确改票数、错误改票数 and 净收益，并使用至少五个随机种子。

8 阶段性结论

项目已经完成候选池构建、执行标注、schema 特征、GPU pairwise 训练、语义表示和选择策略的端到端离线流程，但当前结果没有证明 schema-aware reranker 能提高准确率。这个负结果仍然有价值，因为它排除了一个过于简单的假设：只要把 schema overlap 和冻结语言模型表示加进 MLP，selector 就会自然变好。

目前更合理的判断是，项目的首要瓶颈不是算力，而是有效问题正负对不足，以及候选级语义差异没有被当前特征表示出来。后续工作的重点将从“加入更多特征”转向“构造更有判别力的数据，并直接学习问题与候选逻辑之间的对应关系”。如果经过严格数据库隔离和多随机种子验证后，schema-aware 模型仍不能超过 SQL + execution 模型，那么项目也能够给出一个清晰的研究结论：原 Schema Linker 的硬选择结果对候选重排序没有提供稳定的额外信息。

参考文献

- [1] Xi Victoria Lin, Richard Socher, and Caiming Xiong. Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing. *Findings of EMNLP*, 2020.
- [2] Wenqiang Lei, Weixin Wang, Zhixin Ma, Tian Gan, Wei Lu, Min-Yen Kan, and Tat-Seng Chua. Re-examining the Role of Schema Linking in Text-to-SQL. *EMNLP*, 2020.
- [3] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. *ACL*, 2020.
- [4] Satya Krishna Gorti et al. MSc-SQL: Multi-Sample Critiquing Small Language Models for Text-to-SQL Translation. *NAACL*, 2025.
- [5] Mohammadreza Pourreza et al. CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL. *arXiv:2410.01943*, 2024.
- [6] Boyan Li, Ou Ocean Kun Hei, Yue Yu, and Yuyu Luo. DPC: Training-Free Text-to-SQL Candidate Selection via Dual-Paradigm Consistency. *ACL*, 2026.